Exploring Stock–Housing Market Relationships Using Distributed Big Data Processing

Louis Brennan
Student ID: 23409224

Torin Murray
Student ID: 23381726

Fransisc Pap
Student ID: 23450292

Shane Whelan
Student ID: 23344406

Liang Yu
Student ID: 23363943

COMP30770 — Programming for Big Data
University College Dublin

Code: https://github.com/swhelan123/comp30770-big-data-project

1. Introduction

This project uses two datasets to investigate the relationship between stock market prices and residential property values. The first is the *2000+ Assets (Stock & FX) Historical Price* dataset from Kaggle, comprising 56.8 million rows across 3,762 individual CSV files totalling 3.12 GB. At this scale, a standard single-node SQL pipeline took over five minutes to run on an Apple M4 MacBook Air (24 GB RAM, macOS 26), as detailed in Section 2. The second is the S&P/Case–Shiller U.S. National Home Price Index, a monthly composite tracking national residential property values. The two datasets differ in both structure and granularity: the stock data is spread across thousands of files at daily per-asset resolution, while the housing data is a single monthly time series. Combining them requires temporal alignment and per-ticker aggregation before any statistical analysis can be performed.

The aim of this project is to determine whether month-over-month changes in individual stock prices are correlated with changes in the national housing index. A Hadoop MapReduce pipeline was used to make this analysis practical at the given data scale, processing all assets without requiring the files to be merged in advance.

2. Traditional Data Processing Baseline

2.1 Architecture & Setup

- **Hardware:** Apple M4 MacBook Air, 24 GB RAM, macOS 26. Peak memory during SQL aggregation: ∼18 GB.
- **Database Engine:** MySQL accessed via DataGrip.
- **Pipeline Design:** A sequential pipeline consisting of a single-threaded Python data preparation step followed by four SQL queries, each operating on the full dataset.

2.2 Pipeline Implementation

Step 0: Data Preparation (Sequential I/O). MySQL cannot ingest a directory of separate files directly. A Python script was used to concatenate the 3,761 stock CSVs into a single 3.12 GB file, adding a ticker column from each filename in the process.

```
import pandas as pd, glob, os

folder_path = "./data/1d"
```

```python
all_files = glob.glob(os.path.join(folder_path, "*.csv"))
df_list = []

for file in all_files:
    df = pd.read_csv(file)
    df['Ticker'] = os.path.basename(file).split('.')[0]
    df_list.append(df)

big_df = pd.concat(df_list, ignore_index=True)
cols = ['Ticker', 'Date', 'Adj Close', 'Close', 'High', 'Low', 'Open', 'Volume']
big_df = big_df[cols]
big_df.to_csv('combined_stocks.csv', index=False)
```

```bash
time python3 merge.py
# 99.66s user  9.78s system  93% cpu  1:57.49 total
# Peak memory: ~4 GB
```

Step 1: Temporal Aggregation. The 56.8 M daily stock rows were grouped into monthly averages to match the granularity of the housing index.

```cpp
CREATE TABLE monthly_stocks AS
SELECT
    Ticker,
    DATE_FORMAT(Date, '%Y-%m') AS YearMonth,
    AVG(Close)                 AS Monthly_Avg_Close
FROM stock_prices
GROUP BY Ticker, DATE_FORMAT(Date, '%Y-%m');
-- Runtime: 3m 04.85s  |  Output: 2.7M rows  |  Peak memory: ~18 GB
```

Step 2: Data Integration (Inner Join). Monthly stock averages were joined with the Case–Shiller index on a shared `YYYY-MM` key, keeping only months present in both datasets (1987–2021).

```cpp
CREATE TABLE joined_data AS
SELECT s.Ticker, s.YearMonth, s.Monthly_Avg_Close, h.Housing_Index
FROM monthly_stocks s
JOIN housing_data h ON s.YearMonth = DATE_FORMAT(h.Record_Date, '%Y-%m');
-- Runtime: 1.93s  |  Output: 1.58M rows
```

Step 3: Feature Engineering (Window Functions). Month-over-month percentage change was computed for both series using `LAG` window functions. `NULLIF` prevents division by zero where historical close prices are $0.00.

```cpp
CREATE TABLE feature_engineered_data AS
SELECT Ticker, YearMonth,
    (Monthly_Avg_Close - LAG(Monthly_Avg_Close)
        OVER (PARTITION BY Ticker ORDER BY YearMonth))
    / NULLIF(LAG(Monthly_Avg_Close)
        OVER (PARTITION BY Ticker ORDER BY YearMonth), 0) AS Stock_Pct_Change,
    (Housing_Index - LAG(Housing_Index)
        OVER (PARTITION BY Ticker ORDER BY YearMonth))
    / NULLIF(LAG(Housing_Index)
        OVER (PARTITION BY Ticker ORDER BY YearMonth), 0) AS Housing_Pct_Change
FROM joined_data;
-- Runtime: 33.23s
```

Step 4: Statistical Analysis (Pearson Correlation). The Pearson Correlation Coefficient (r) was computed per ticker. Tickers with fewer than 12 overlapping months were excluded.

```cpp
CREATE TABLE final_correlation AS
SELECT Ticker, COUNT(*) AS Months_Correlated,
    (COUNT(*) * SUM(Stock_Pct_Change * Housing_Pct_Change)
```

```sql
        - SUM(Stock_Pct_Change) * SUM(Housing_Pct_Change))
    / NULLIF(SQRT(
        (COUNT(*) * SUM(Stock_Pct_Change * Stock_Pct_Change)
            - POW(SUM(Stock_Pct_Change), 2))
      * (COUNT(*) * SUM(Housing_Pct_Change * Housing_Pct_Change)
            - POW(SUM(Housing_Pct_Change), 2))
    ), 0) AS Pearson_Correlation
FROM feature_engineered_data
WHERE Stock_Pct_Change IS NOT NULL AND Housing_Pct_Change IS NOT NULL
GROUP BY Ticker HAVING COUNT(*) >= 12
ORDER BY Pearson_Correlation DESC;
-- Runtime: 1.57s  |  Output: 2,901 rows
```

2.3 Execution Log & Performance Metrics

Table 1: Traditional pipeline execution metrics (Apple M4, 24 GB RAM).

Step	Operation	Output	Time	Peak Memory
0	Data Prep (`merge.py`)	1 CSV (3.12 GB)	1 m 57 s	∼4 GB
1	Temporal Aggregation	2.7 M rows	3 m 05 s	∼18 GB
2	Inner Join	1.58 M rows	1.93 s	—
3	Feature Engineering	1.58 M rows	33.23 s	—
4	Statistical Analysis	2,901 rows	1.57 s	—
Total (End-to-End)		**2,901 correlations**	**∼5 m 39 s**	**∼18 GB peak**

2.4 Baseline Bottlenecks

Steps 0 and 1 together account for roughly 89% of total runtime, while Steps 2–4 complete in under 37 seconds combined. These two steps are the primary targets for optimisation and are discussed in Section 3.1.

3. MapReduce Optimisation

3.1 Bottleneck Identification & Expectations

Based on the timings in Section 2.3, Steps 0 and 1 are the bottlenecks, accounting for 89% of total runtime (∼5 minutes).

Step 0 (Sequential I/O): The need to pre-merge 3,761 files is a limitation specific to relational databases. Hadoop's `FileInputFormat` reads a directory of files as a single input, so this step is unnecessary and its 117-second cost is eliminated entirely.

Step 1 (Single-Node Grouping): Grouping 56.8 million rows on a single core is the main compute bottleneck. MapReduce assigns one file per mapper, so all 3,762 CSVs are processed in parallel. The shuffle phase then groups records by ticker before they reach the reducer, replacing the sequential `GROUP BY` scan.

Expectation: Removing Step 0 and parallelising Step 1 across two datanodes should reduce the combined ingestion and aggregation time by a factor of 3 to 5, depending on container availability and HDFS replication overhead.

3.2 MapReduce Solution

The implementation parallelises Steps 0–1 and combines Steps 2–4 into the reduce phase, avoiding any intermediate disk writes. The full source (`StockHousingCorrelation.java`) is on GitHub; the key logic is presented below.

Key–Value Design.

- **Map output:** `key = ticker (Text)`, `value = "STOCK,YYYY-MM,closePrice (Text)"`
- **Reduce input:** all values for a given ticker, grouped and sorted by Hadoop's shuffle
- **Reduce output:** `key = ticker (Text)`, `value = "n_months\tpearson_r (Text)"`

Map Function.

The mapper:

- Processes one stock CSV per task (3,762 tasks in total).
- Reads the ticker from the filename (e.g. `AAPL.csv` → `AAPL`), since column 0 of each file is `Date`.
- Emits one key–value pair per valid row: `(ticker, "STOCK,YYYY-MM,closePrice")`.

```
@Override
protected void setup(Context context) throws IOException, InterruptedException {
    String filename = ((FileSplit) context.getInputSplit()).getPath().getName();
    ticker = filename.replace(".csv", "");
}

@Override
public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
    String[] parts = value.toString().trim().split(",");
    if (parts[0].equalsIgnoreCase("Date")) return;
    if (parts.length >= 3 && !parts[2].trim().isEmpty()) {
        try {
            double closePrice = Double.parseDouble(parts[2].trim());
            String yearMonth = parts[0].trim().substring(0, 7);
            outKey.set(ticker);
            outValue.set("STOCK," + yearMonth + "," + closePrice);
            context.write(outKey, outValue);
        } catch (NumberFormatException e) { }
    }
}
```

Reduce Function.

For each ticker, the reducer:

1. Computes monthly average close prices (equivalent to SQL Step 1).
2. Sorts months chronologically.
3. Computes month-over-month percentage change for both series (equivalent to SQL Step 3).
4. Calculates the Pearson Correlation Coefficient (equivalent to SQL Step 4).

The housing index is read directly from HDFS in `setup()` before any `reduce()` calls, which avoids routing it through the shuffle. Tickers with fewer than 12 overlapping months are dropped. The full reducer is on GitHub; the Pearson calculation is shown below.

```
@Override
protected void setup(Context context) throws IOException, InterruptedException {
    FileSystem fs = FileSystem.get(context.getConfiguration());
    BufferedReader br = new BufferedReader(
        new InputStreamReader(fs.open(new Path("/input/correlation/CSUSHPINSA.csv"))));
    String line;
    while ((line = br.readLine()) != null) {
        String[] parts = line.split(",");
        if (parts.length == 2 && !parts[0].trim().equalsIgnoreCase("observation_date"))
            housingData.put(parts[0].trim().substring(0, 7),
                            Double.parseDouble(parts[1].trim()));
    }
    br.close();
}

// Pearson correlation over month-over-month pct changes
double numerator = (n * sumXY) - (sumX * sumY);
```

```
double denominatorSq =
    ((n * sumX2) - (sumX * sumX)) * ((n * sumY2) - (sumY * sumY));
if (denominatorSq > 0) {
    double correlation = numerator / Math.sqrt(denominatorSq);
    outValue.set(n + "\t" + String.format("%.6f", correlation));
    context.write(key, outValue);
}
```

The job was configured with a single reducer (`mapreduce.job.reduces=1`) and submitted to YARN (`mapreduce.framework.name=yarn`) for execution across two datanode containers.

3.3 MapReduce Results

The job ran on a Dockerised Hadoop cluster (one namenode, two datanodes, one ResourceManager) on the same Apple M4 MacBook Air. YARN submission was confirmed by the job ID and ResourceManager log:

```
INFO client.RMProxy: Connecting to ResourceManager at resourcemanager/172.18.0.5:8032
INFO mapreduce.Job: Running job: job_1774108324741_0001
```

Table 2: MapReduce vs. traditional baseline: performance comparison.

Step	Traditional (SQL)	MapReduce (YARN)
Step 0: Pre-processing (file merge)	1 m 57 s	0 s (eliminated)
Step 1: Aggregation / Map phase	3 m 05 s	∼55 s
Steps 2–4: Join, features, stats	36.73 s	(inline in reducer, no materialisation)
Total end-to-end	**5 m 39 s**	**1 m 27 s**
Speedup factor	—	**3.9**×
Map input records	56.8 M	56.8 M
Peak memory	∼18 GB	∼2 GB (per container)
Output records	2,901	2,899

3.4 Critical Analysis

The results largely match expectations, with an end-to-end speedup of **3.9**×.

1. **I/O Efficiency:** The largest gain came from removing Step 0. Because `FileInputFormat` reads the 3,762-file directory in parallel, the 117-second Python merge was bypassed entirely, accounting for 35% of the total baseline time.
2. **Scaling vs. Overhead:** Despite using two datanodes, the aggregation phase improved by roughly 3.4× rather than the theoretical 2× per-node figure. This is partly because the per-file parallelism across 3,762 mappers is more significant than the two-node split, but also because JVM container startup and shuffle network I/O add overhead that grows less significant at larger cluster sizes.
3. **Data Skew:** Some tickers had substantially more data than others. AAPL and ABT both have 421 months of records, while many recently listed assets have fewer than 50. This meant a small number of map tasks took noticeably longer than average, limiting the parallel efficiency.
4. **Correctness:** Output was cross-checked against the SQL results. Both pipelines use the same method and produce consistent rankings, with small numerical differences due to floating-point precision between Java `double` and MySQL decimal types. An earlier version passed housing data through the shuffle using a sentinel key (`000_HOUSING`), but this produced no output in

local mode because key ordering within a single JVM is not guaranteed. Loading the housing data in `setup()` resolved this.

The MapReduce approach removed both the file I/O bottleneck and the single-node aggregation constraint that dominated the SQL pipeline runtime.

4. Results

The MapReduce job produced Pearson Correlation Coefficients for **2,899 tickers**, cross-checked against the SQL baseline. Figure 1 shows the two indices over time. Both trend upward across the full period, but diverge at several points. The analysis here measures month-over-month changes rather than index levels, which are considerably noisier and explain why the correlation coefficients are low even where the long-run trends appear similar.

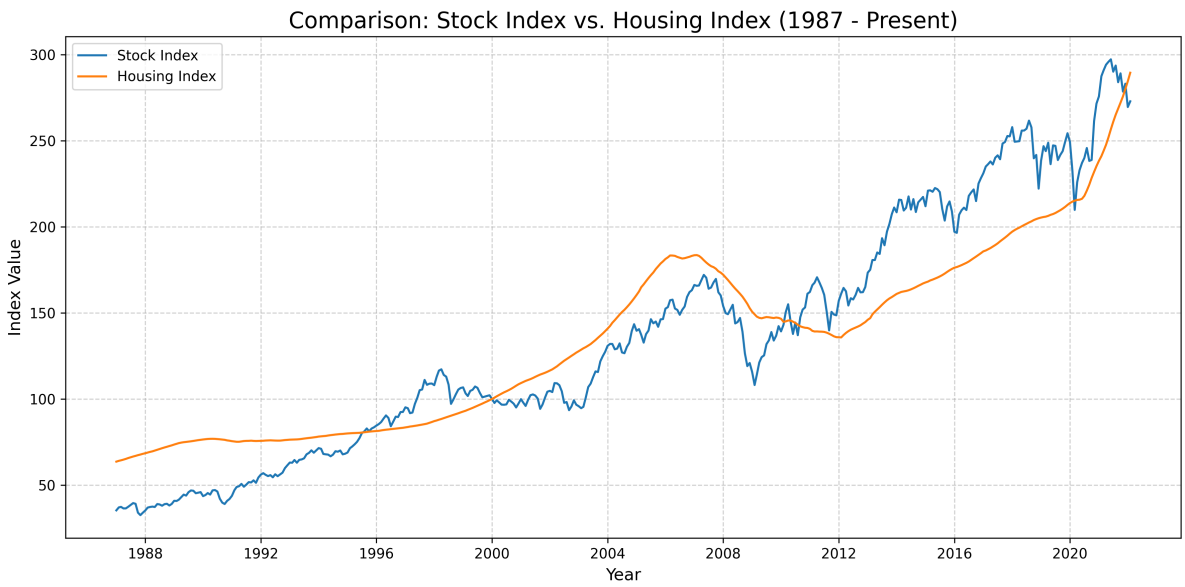

Figure 1: S&P 500 stock index vs. the Case–Shiller U.S. National Home Price Index, 1987–present. Both indices trend upward overall, but diverge during the 2008 financial crisis and again after 2020.

Table 3 shows the ten most positively correlated assets, restricted to tickers with at least 100 months of overlapping data.

Table 3: Top 10 most positively correlated assets with the U.S. housing index (≥100 overlapping months).

Ticker	Months Compared	Pearson r
WSBF	196	0.3553
FTGC	100	0.3319
HMNF	332	0.3155
HBP	266	0.2874
MVBF	114	0.2838
CELH	181	0.2660
GBPJPY-X	218	0.2642
CME	230	0.2640
AAXJ	162	0.0990
ABCB	333	0.0985

Correlation values are low across all 2,899 assets. The highest value among tickers with

at least 100 months of data is $r \approx 0.36$, and most fall below $|r| < 0.2$. This suggests that at monthly granularity, individual stock returns and housing price changes are largely independent, even though both are influenced by broader macroeconomic conditions. As Figure 1 shows, the two markets responded differently to the 2008 financial crisis: housing peaked in 2006 and fell through 2012, while equity markets recovered more quickly. A similar divergence occurred after 2020. These periods reduce the month-over-month correlation precisely when the relationship between markets is most economically interesting. Sector-specific analysis, for example focusing on REITs or homebuilders, or using longer time lags, would likely produce stronger results. Note that FX pairs such as `GBPJPY-X` appear in the output since they were stored in the same input directory as the equities, and would need to be filtered out for any equity-specific analysis.

5. Conclusion

This project built and compared two pipelines for computing Pearson correlations between individual stock returns and the U.S. housing index across 3,762 assets and 56.8 million records. The SQL baseline took 5 m 39 s with a peak memory of ∼18 GB, with 89% of that time spent on file merging and aggregation. The Hadoop MapReduce implementation removed the merge step entirely and distributed the aggregation across 3,762 parallel map tasks, completing in 1 m 27 s and reducing peak per-container memory to ∼2 GB, a **3.9× speedup**.

The gap between the observed and theoretical speedup reflects JVM startup costs, shuffle I/O overhead, and skew from tickers with long data histories. These effects become proportionally smaller at larger cluster sizes.

On the analytical side, month-over-month stock returns show weak correlation with housing price changes for the vast majority of assets ($|r| < 0.2$ for most, maximum reliable $r \approx 0.36$). The two markets share broad long-run trends but diverge at major economic events, which suppresses short-term correlation. This is consistent with the cross-asset correlation literature at short timescales, and suggests that longer lags or sector-level groupings would be more informative.